

Google Developer Group
Universitas Sriwijaya

Learning 12

Introduction to Supabase & Authentication

Using Supabase as a BaaS for React applications



```
const org = interbyOrg ? study.lead_organization === interbyOrg : C  
if (Status = filterByStatus ? study.status === filterByStatus : true  
  (matchStatus) {
```

```
function filterStudies({ studies, filterByOrg = null }) {  
  return studies.filter(study => {  
    if (filterByOrg) {  
      return study.lead_organization === filterByOrg
```

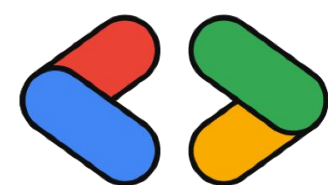
What is Supabase?

Supabase is an open-source Backend-as-a-Service (BaaS)

It provides backend features without building a backend server manually.

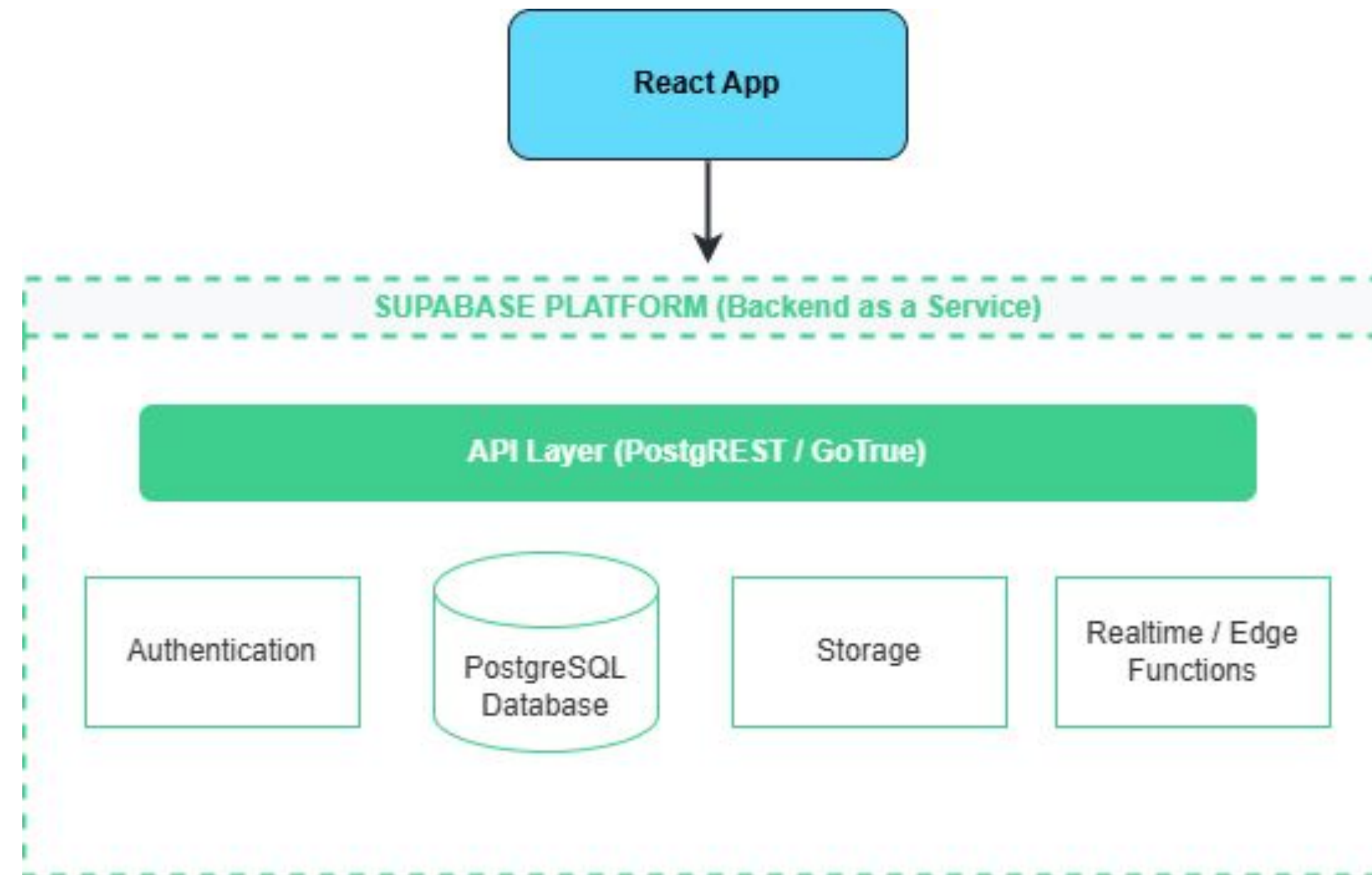
Core features:

- PostgreSQL database
- Authentication
- Storage
- Real-time API
- Auto-generated REST API

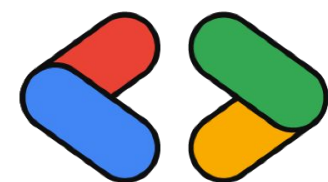


Google Developer Group
Universitas Sriwijaya

Supabase Architecture



Developers interact with Supabase using Supabase Client SDK



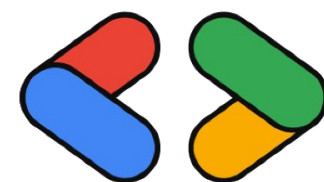
Supabase Uses PostgreSQL

Supabase uses PostgreSQL as its database.

PostgreSQL is a powerful relational database system.

Benefits:

- reliable and scalable
- supports SQL queries
- strong ecosystem
- widely used in production systems



Google Developer Group
Universitas Sriwijaya

Creating a Supabase Project

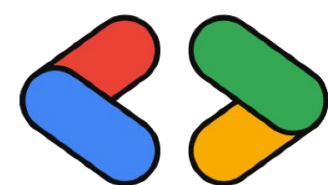
Steps:

1. Create account at Supabase
2. Create a new project
3. Get project credentials

Important credentials:

Project URL
Anon Public Key

These keys allow the frontend application to connect to Supabase.



Google Developer Group
Universitas Sriwijaya

Installing Supabase Client

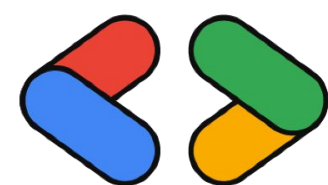
Install Supabase SDK

```
npm install @supabase/supabase-js
```

Create a client configuration

```
import { createClient } from "@supabase/supabase-js"

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)
```



Google Developer Group
Universitas Sriwijaya

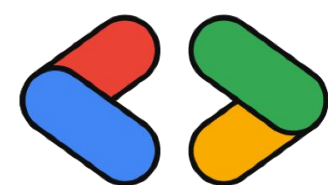
Register User

Supabase provides built-in authentication.

```
const { data, error } = await supabase.auth.signUp({  
  email,  
  password  
})
```

Supabase automatically handles:

- password hashing
- user management
- authentication tokens

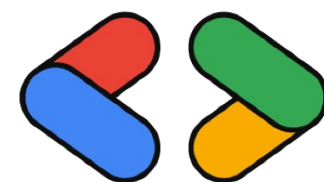


Login User

```
const { data, error } = await supabase.auth.signInWithPassword({  
  email,  
  password  
})
```

If successful, Supabase returns:

- user information
- authentication session



Getting Current Session

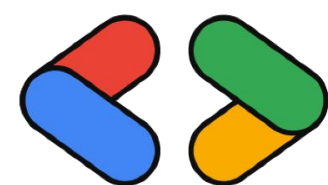
Applications often need to check whether a user is logged in.

```
const { data, error } = await supabase.auth.signInWithPassword({  
  email,  
  password  
})
```

The session contains:

- user data
- access token
- authentication status

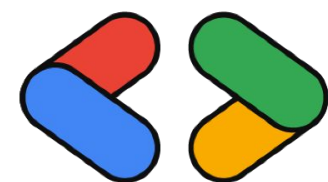
This allows the app to protect pages or personalize UI.



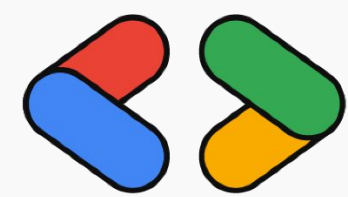
Key Takeaways

Rules:

- Supabase is a Backend-as-a-Service platform
- It uses PostgreSQL as the database
- Supabase provides built-in authentication
- React apps connect using the Supabase Client SDK



Google Developer Group
Universitas Sriwijaya



Google Developer Group
Universitas Sriwijaya

Thank You !

```
filterByOrg = filterByOrg ? study.lead_organization === filterByOrg : true  
filterByStatus = filterByStatus ? study.status === filterByStatus : true  
filterByStatus = filterByStatus ? study.status === filterByStatus : true  
filterByStatus = filterByStatus ? study.status === filterByStatus : true
```

```
function filterStudies({ studies, filterByOrg, filterByStatus }) {  
  return studies.filter(study => {  
    return filterByOrg === study.lead_organization &&  
    filterByStatus === study.status
```